Task 1: Web App QA & Debug Report

Automation & QA Developer Skills Assessment | Candidate: Bapi Baidya

Overview & Methodology:
An end-to-end quality assurance and diagnostic audit was executed across the Conduit RealWorld web application (Angular 18 SPA + REST API at https://api.realworld.show/api). Testing exercised core user journeys: user registration, session management, content authoring (CRUD), tag ingestion, and network failure states. The audit surfaced critical functional regressions, validation bypasses, and security smells detailed below.

Identified Issues (Standard QA Template)

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Duplicate Registration Overwrites Identity & Mints New Session Token	1. POST /api/users with new user credentials (gets 201 + Token A). 2. Send identical POST /api/users with exact same email & username. 3. Observe response status & token.	Expected: 422 Unprocessable Entity ('email/username already taken'). Actual: 201 Created with a newly generated JWT token, bypassing identity conflict checks.	**Critical**	Missing unique constraint validation on email & username columns in user registration service.
2	Whitespace Title Accepted, Producing Empty Slug & Breaking Routing	1. Log in and open /editor. 2. Input whitespace spaces (' ') into Title field. 3. Provide valid description and body; submit. 4. Observe redirected URL.	Expected: Validation error rejecting empty/whitespace title. Actual: 201 Created with slug: ''. Browser routes to /article/ (collides with base path; article is orphaned).	**High**	Validation checks truthiness without trimming (if (title)); slugify regex strips all spaces to empty string without fallback ID.
3	Production Build Exposes Global Debug Object Leaking Active JWT Token	1. Navigate to demo.realworld.show and log into account. 2. Open DevTools (F12) -> Console. 3. Run window.__conduit_debug__.getToken().	Expected: Production build strips debug utilities; window.__conduit_debug__ is undefined. Actual: Prints active plain-text JWT token and user authentication state globally.	**High**	Developer debug helper in main bundle was not gated behind !environment.production compiler flags.
4	Absence of Essential HTTP Security Headers Across Edge Gateways	1. Send HEAD / GET request to demo.realworld.show or REST API. 2. Inspect response headers in DevTools Network tab or curl -I.	Expected: Standard security headers enforced (CSP, X-Frame-Options, X-Content-Type-Options, HSTS). Actual: All major security headers are completely missing from response.	Medium	Reverse proxy / cloud ingress gateway deployed without security header middleware (e.g. Helmet.js).
5	Tag Input Permits Unsanitized Raw HTML / Script Payloads	1. In /editor, enter '<script>alert(1)</script>' in 'Enter tags' and press Enter. 2. Publish article. 3. Inspect JSON response payload and home feed tags.	Expected: Tag input sanitizes characters or enforces alphanumeric whitelist. Actual: Raw HTML accepted, persisted in database, and returned verbatim in tagList array.	Medium	Tag handler only checks for non-empty string; lacks regex whitelist validation and HTML entity escaping.

Root-Cause Analysis (RCA): Issue #2 - Empty Slug from Whitespace Title

1. What is Happening: When an author creates an article whose title consists solely of whitespace characters, the backend API accepts the payload with HTTP 201 Created and persists the record with an empty slug (""). Consequently, the client router navigates to https://demo.realworld.show/article/, colliding with parent route patterns. The article becomes untargetable: it renders as a blank feed card, cannot be retrieved by slug, and fails subsequent PUT and DELETE operations with 404/405 errors because REST calls hit the collection root instead of an individual entity.

2. Why it Occurs (Technical Root Cause): In the backend request handler, title validation relies on loose existence checks (if (!title)) rather than asserting non-whitespace length (if (!title || title.trim().length === 0)). Following validation, the slugification utility applies regex character substitution (title.toLowerCase().replace(/[^a-z0-9]+/g, '-').replace(/^-|-$/g, '')), which reduces whitespace strings to an empty string. Because neither the database schema nor ORM model declares a minimum length constraint on the slug column, the empty slug passes through and commits directly to persistent storage.

3. How to Fix It (Engineering Action Plan): First, enforce strict input validation at the schema boundary using Zod/Joi requiring z.string().trim().min(3).max(120) on article titles. Second, refactor the slugification utility with a defensive fallback: if the transformed slug length is 0, generate an automated identifier such as 'untitled-' concatenated with an 8-character nanoid. Finally, apply a database-level CHECK constraint (CHECK (length(trim(slug)) > 0)) and unique index to guarantee storage integrity.